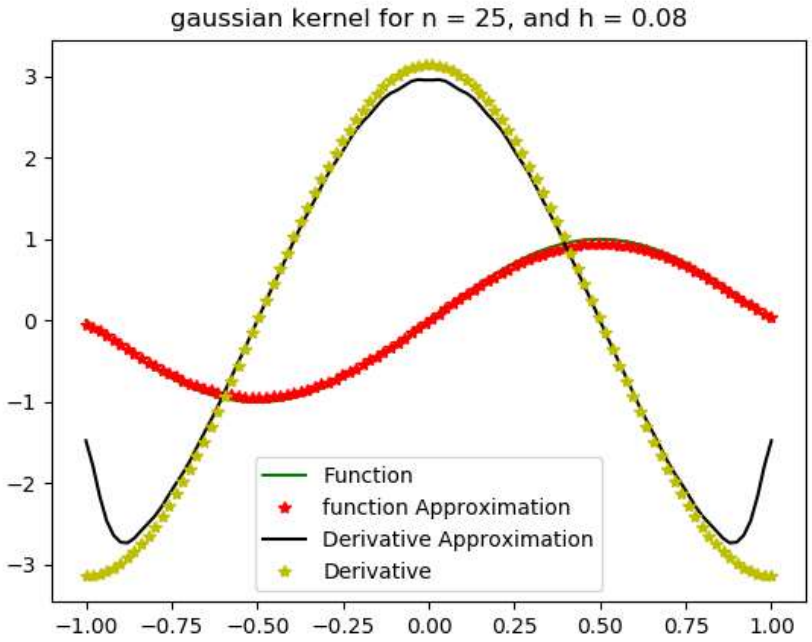
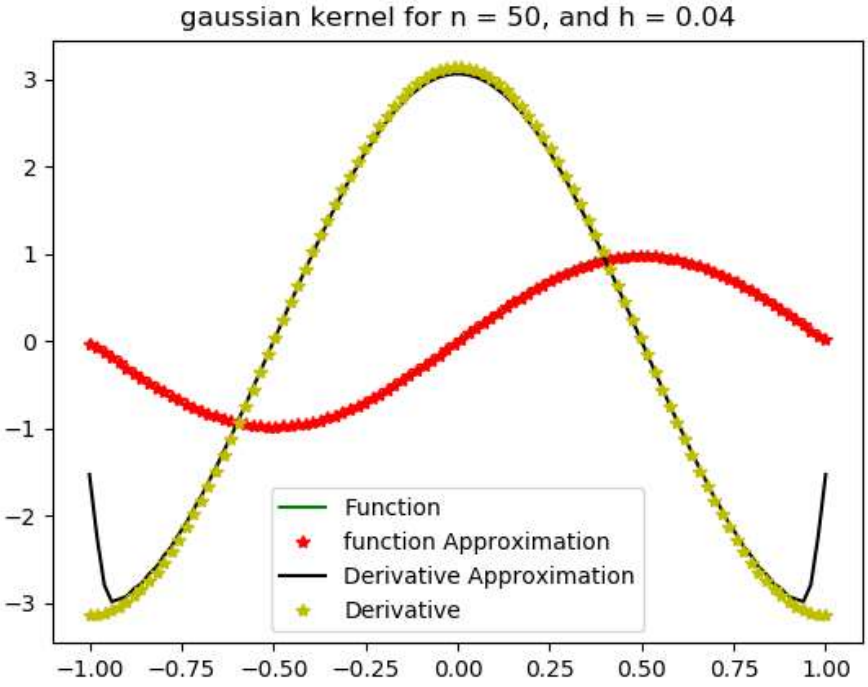
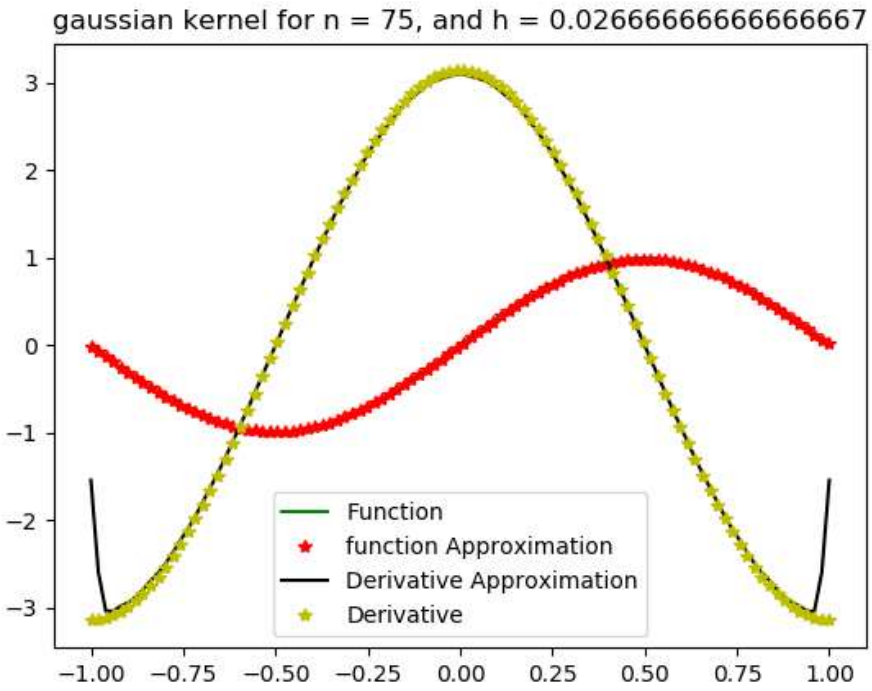
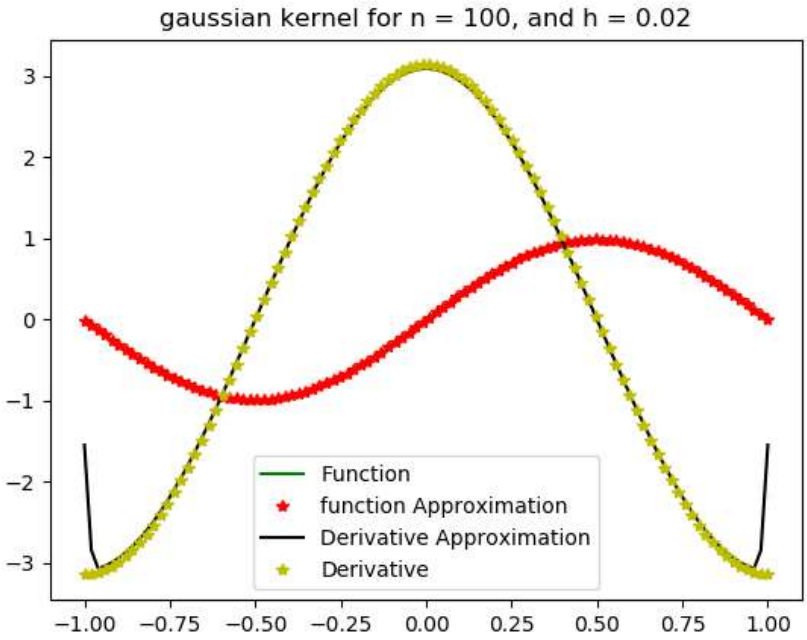
For 1D case

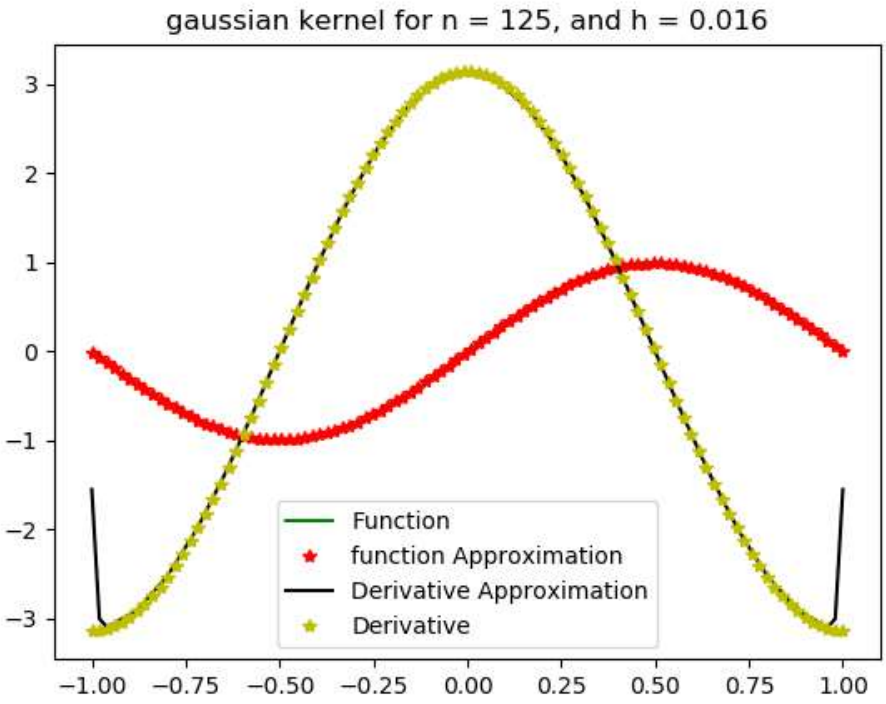
1) **Gaussian kernel**

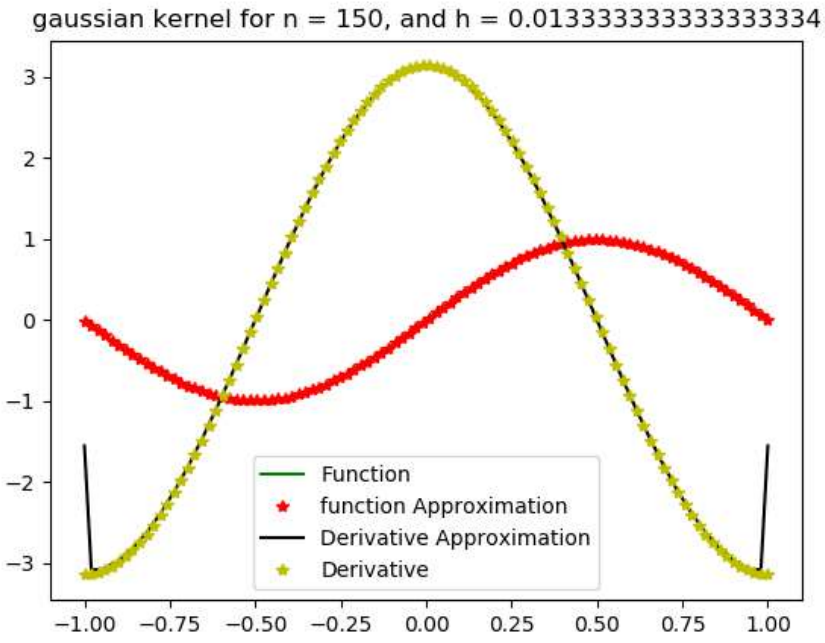




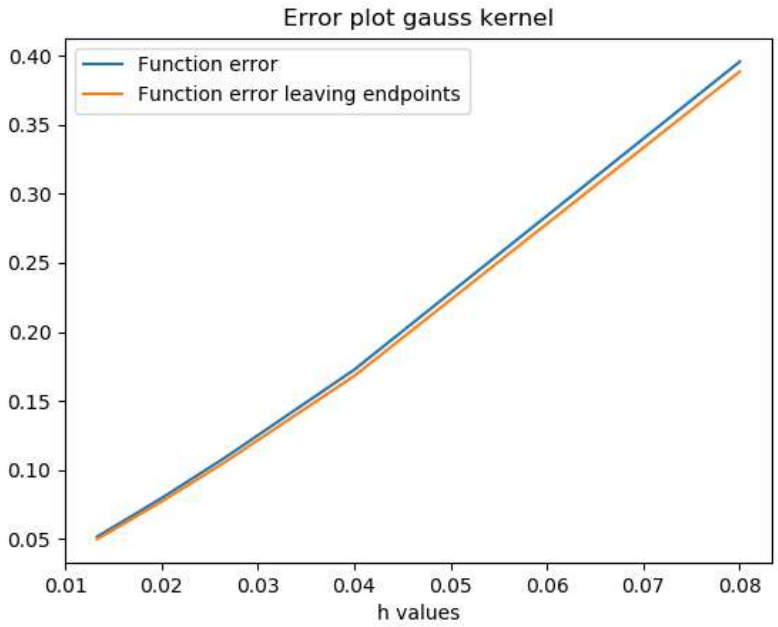

gaussian kernel for n = 75, and h = 0.02666666666666667
Function
function Approximation
Derivative Approximation
Derivative
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

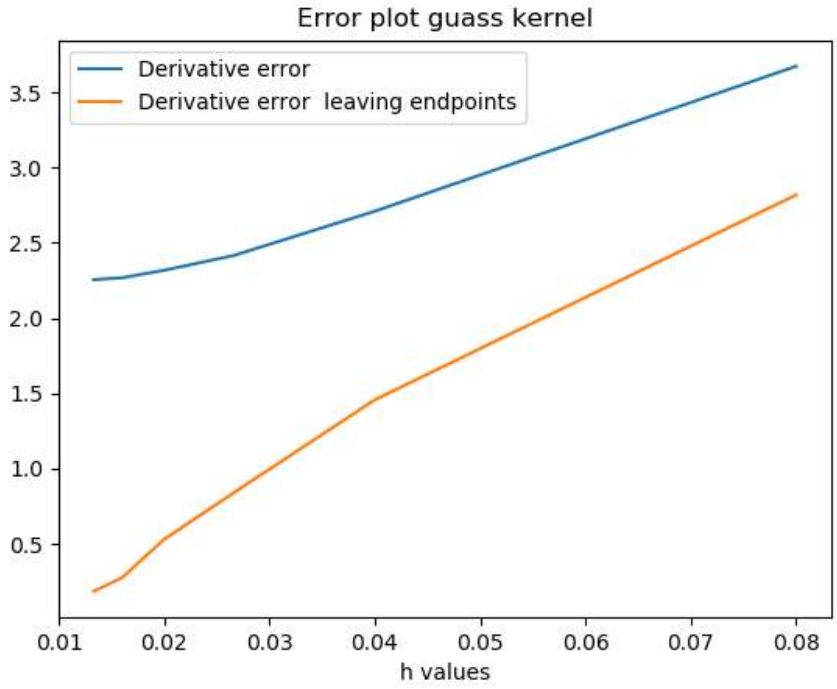

gaussian kernel for n = 100, and h = 0.02
Function
function Approximation
Derivative Approximation
Derivative
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00


gaussian kernel for n = 125, and h = 0.016
Function
function Approximation
Derivative Approximation
Derivative
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

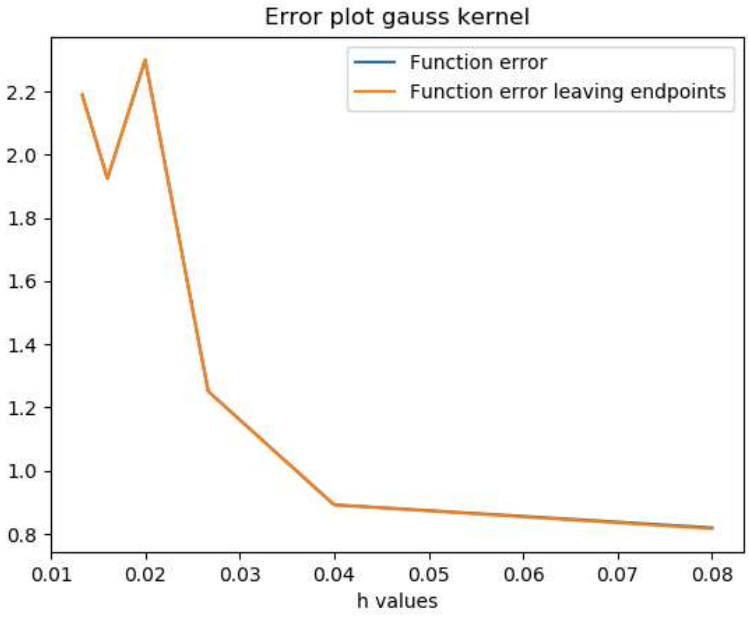

gaussian kernel for n = 150, and h = 0.013333333333333334
Function
function Approximation
Derivative Approximation
Derivative
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

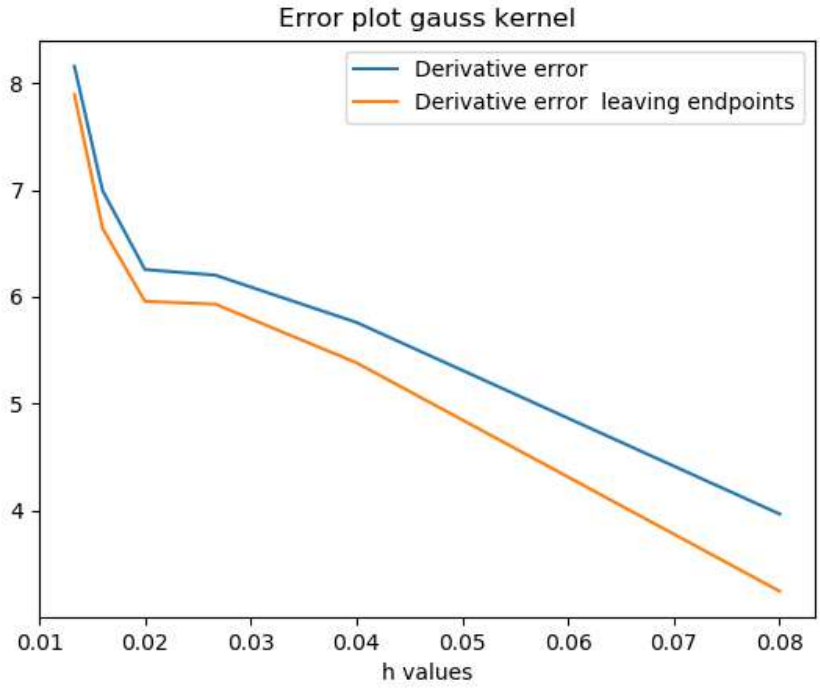
Errors plot for Gaussian kernel without noise in particle position



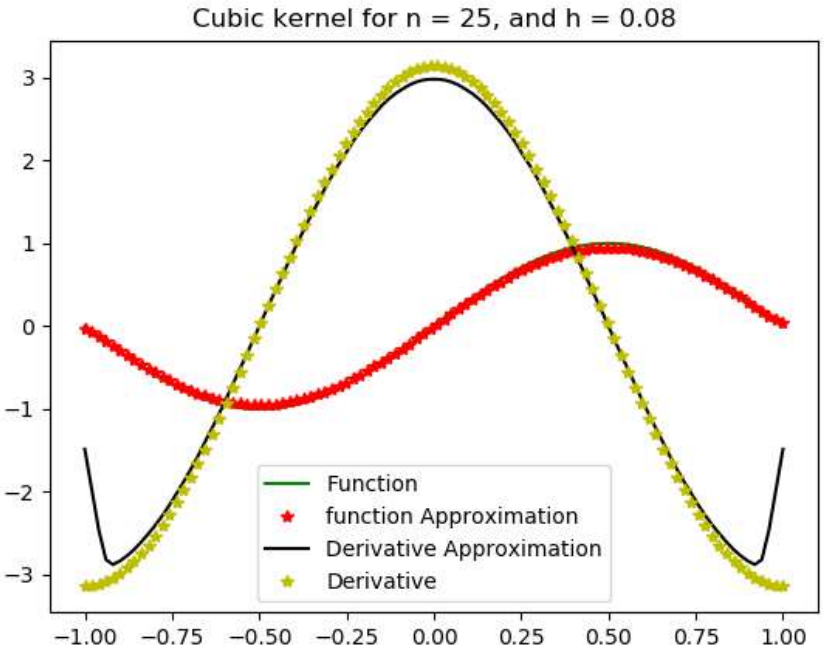


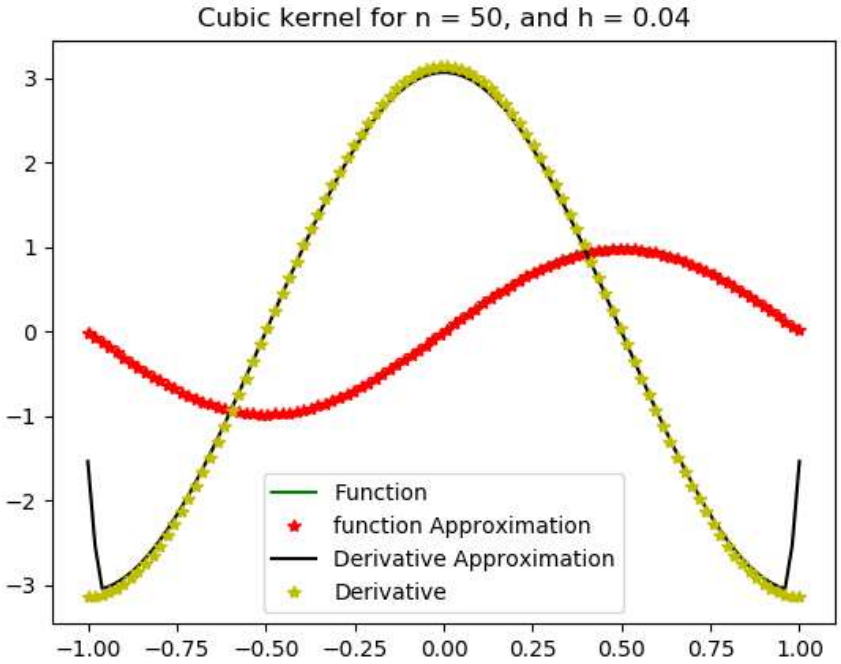
Errors plot for Gaussian kernel with random noise in particle position

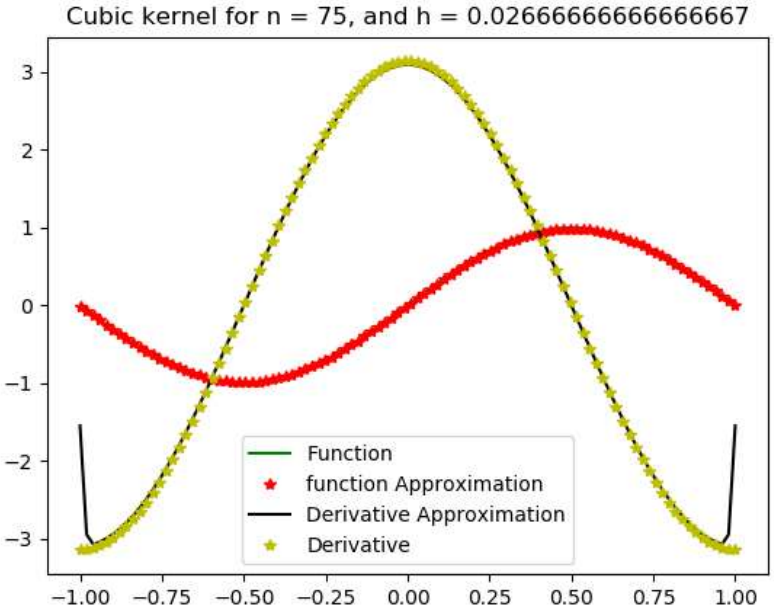


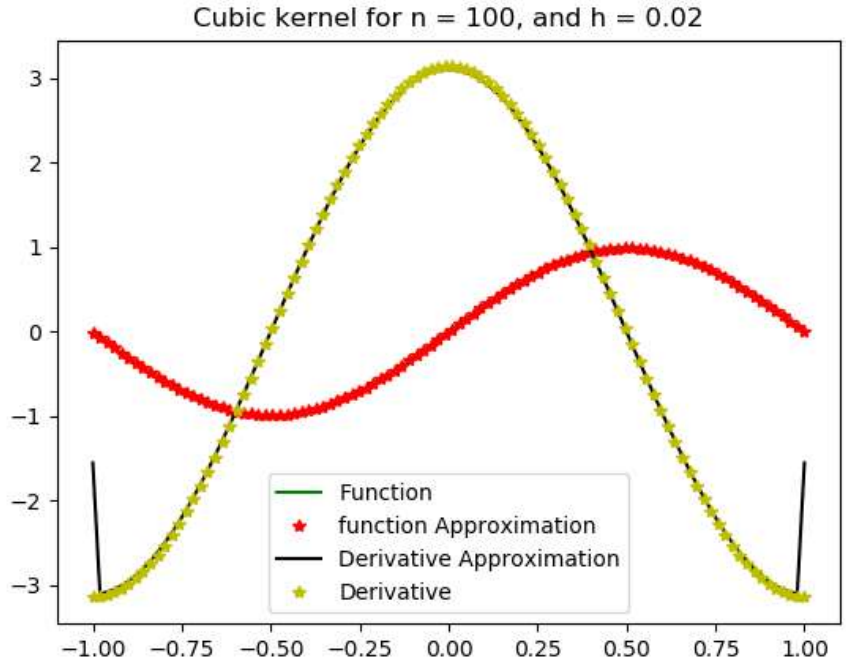


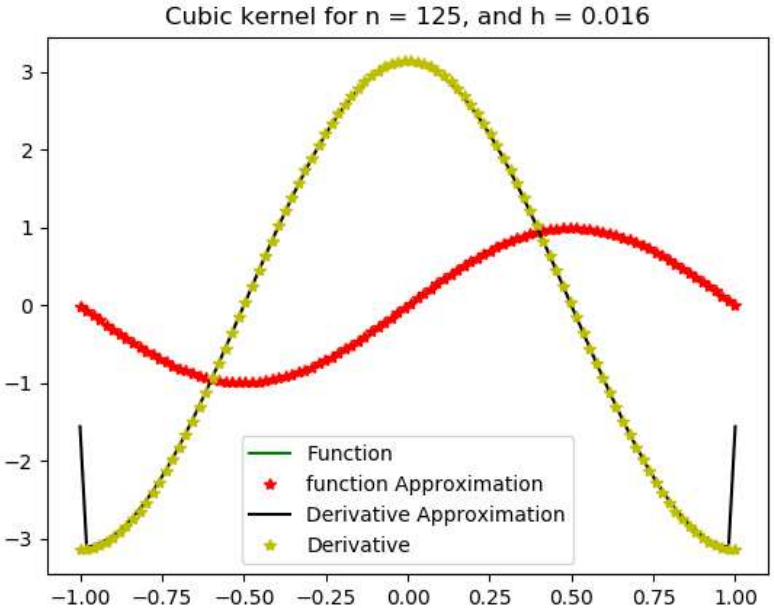
2) Cubic spline kernel

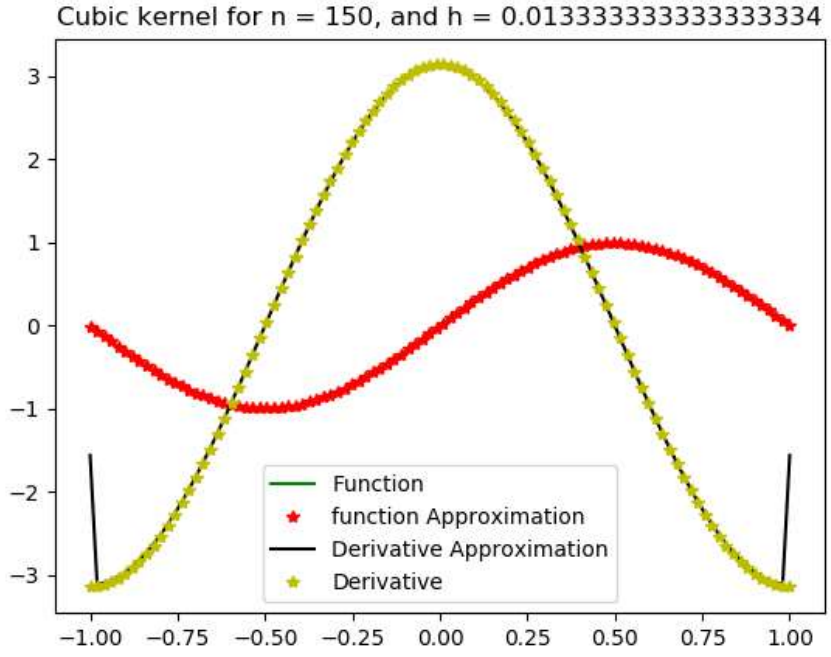




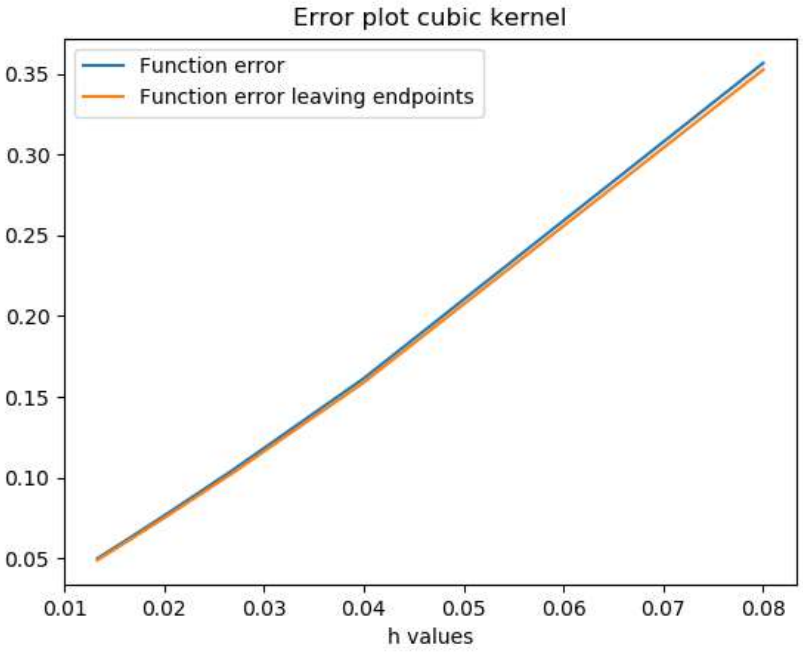

Cubic kernel for n = 75, and h = 0.02666666666666667
Function
function Approximation
Derivative Approximation
Derivative
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

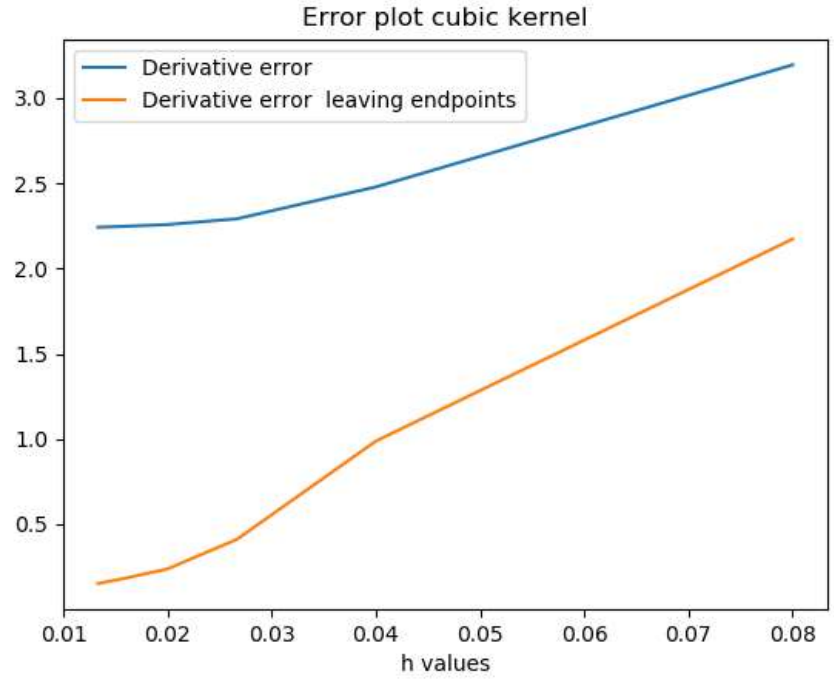

Cubic kernel for n = 100, and h = 0.02
Function
function Approximation
Derivative Approximation
Derivative
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00


Cubic kernel for n = 125, and h = 0.016
Function
function Approximation
Derivative Approximation
Derivative
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00


Cubic kernel for n = 150, and h = 0.013333333333333334
Function
function Approximation
Derivative Approximation
Derivative
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

Errors plot for Cubic kernel



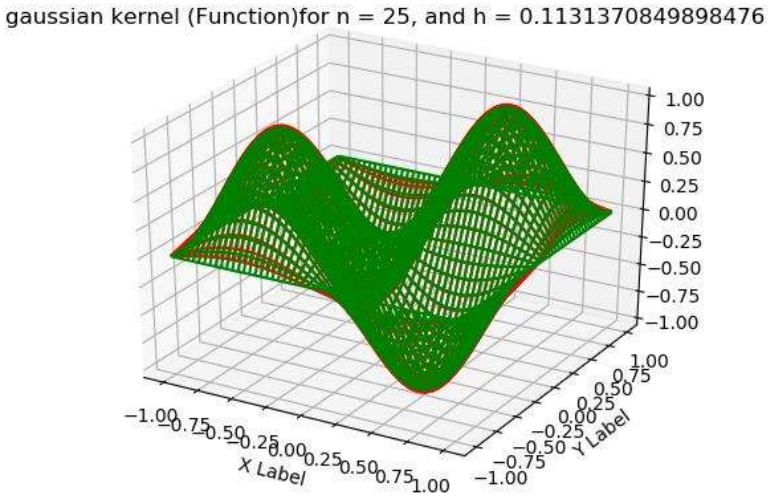


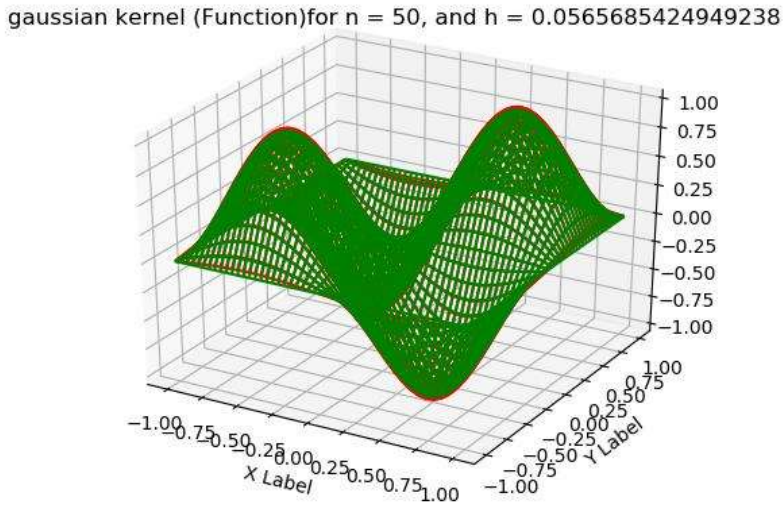
For 2D case

I have included plots for Gaussian kernel only the plots for cubic are also similar

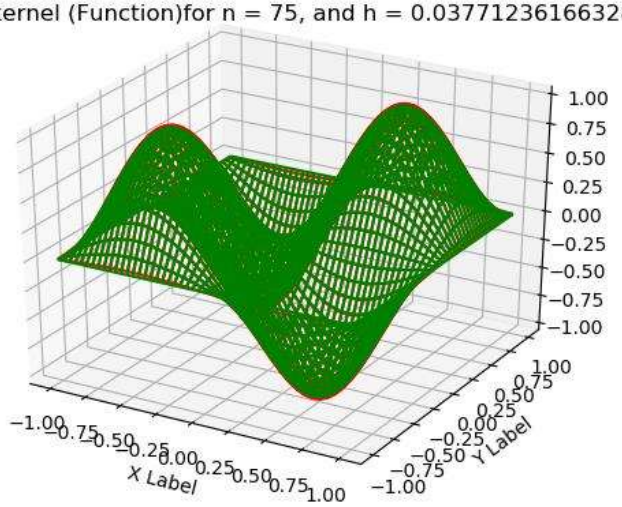
The green plots are the actual function and red ones are the approximations

1) **Gaussian kernel**
   a) **Function**

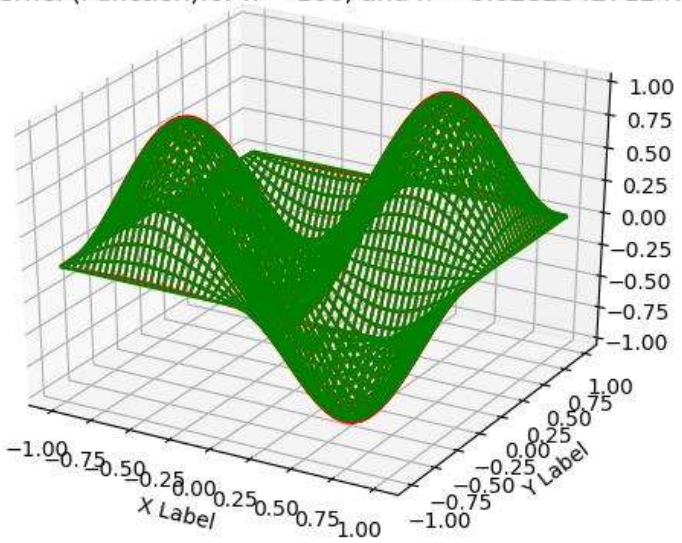


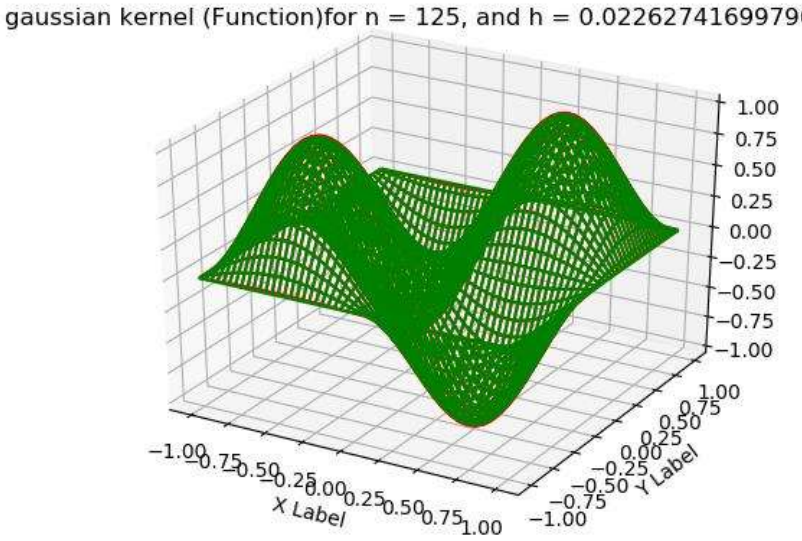


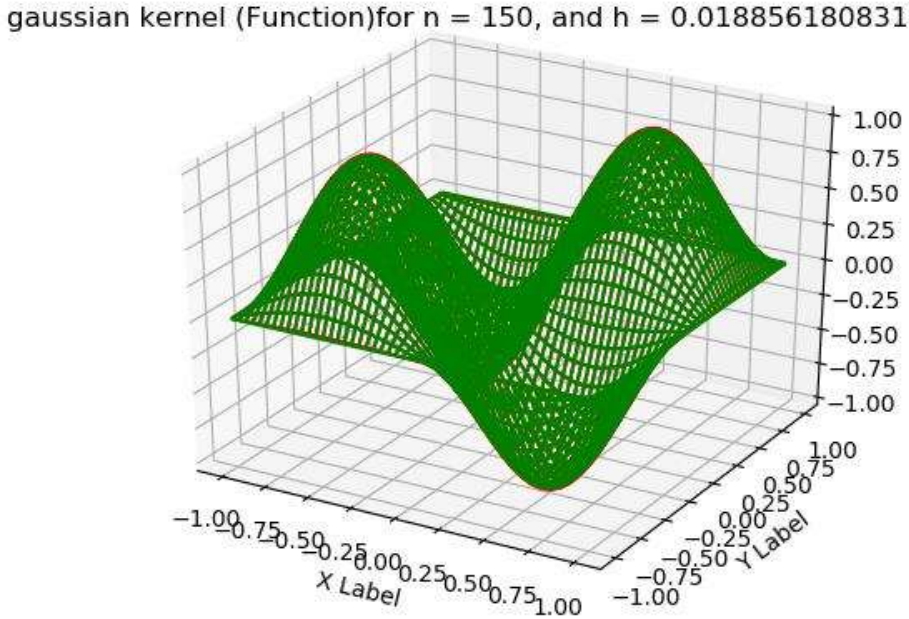
gaussian kernel (Function)for n = 75, and h = 0.03771236166328254



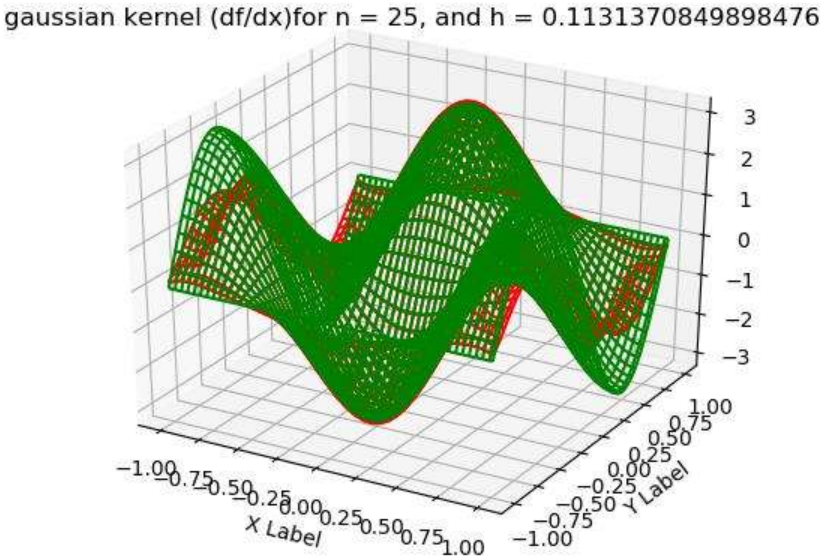
gaussian kernel (Function)for n = 100, and h = 0.0282842712474619

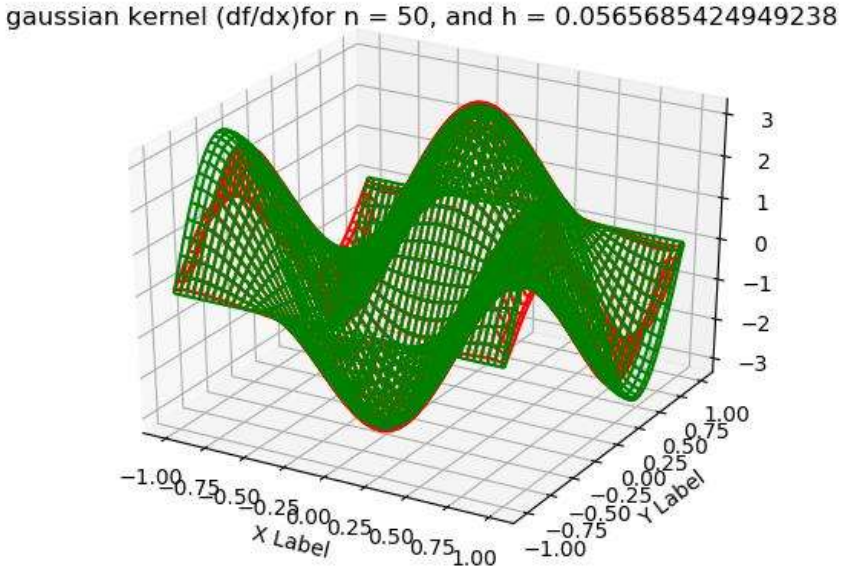



gaussian kernel (Function)for n = 125, and h = 0.02262741699796952
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
X Label
Y Label

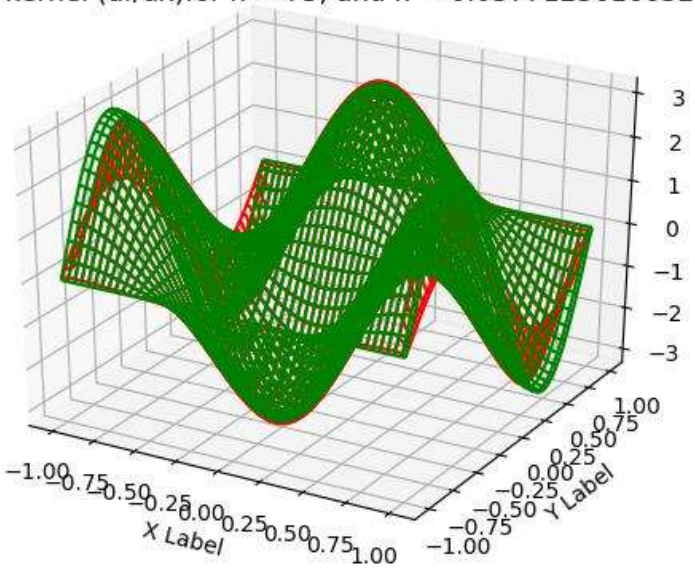

gaussian kernel (Function)for n = 150, and h = 0.01885618083164127
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
X Label
Y Label

b) Its df/dx

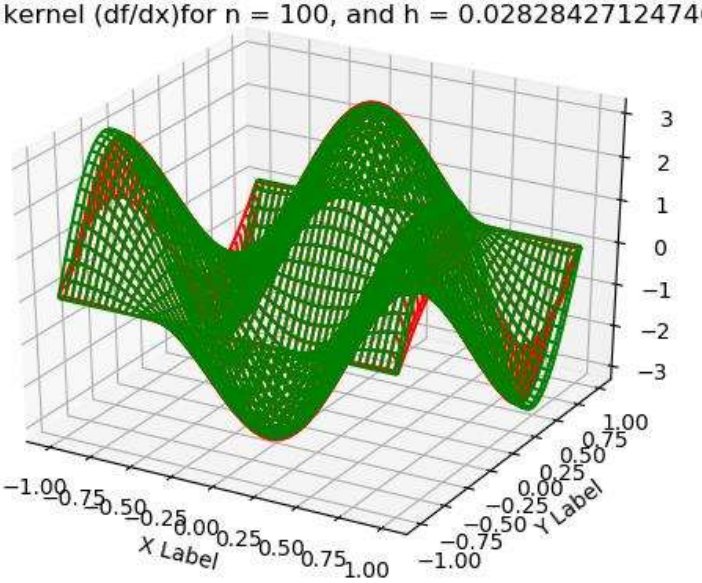

gaussian kernel (df/dx)for n = 25, and h = 0.1131370849898476
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
X Label
Y Label


gaussian kernel (df/dx)for n = 50, and h = 0.0565685424949238
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
X Label
Y Label

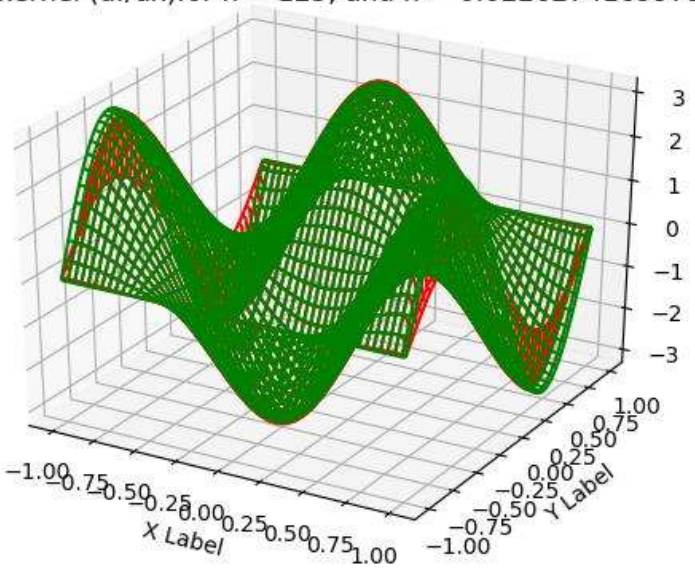
gaussian kernel (df/dx)for n = 75, and h = 0.037712361663282 54



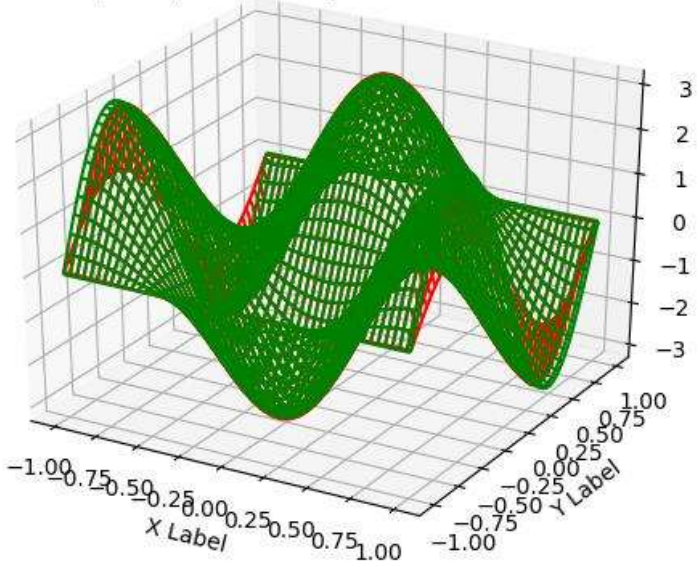
gaussian kernel (df/dx)for n = 100, and h = 0.0282842712474619



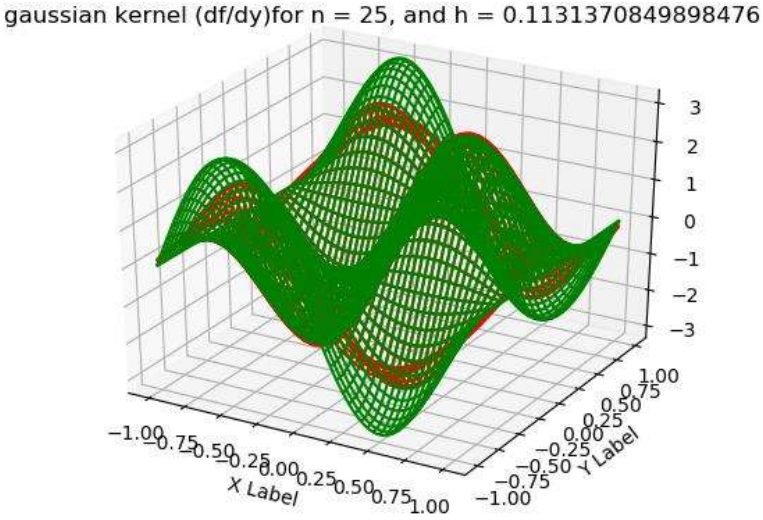
gaussian kernel (df/dx)for n = 125, and h = 0.022627416997969 52

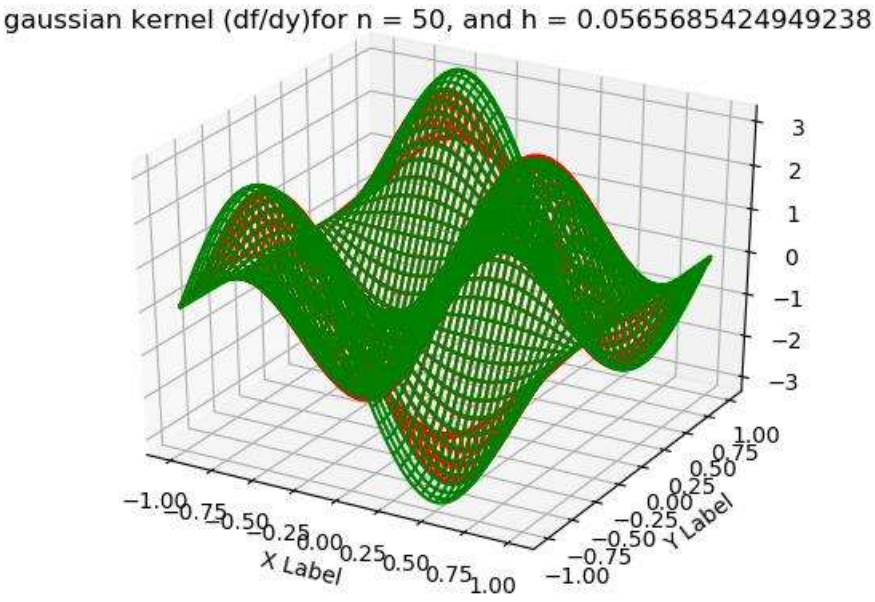


gaussian kernel (df/dx)for n = 150, and h = 0.01885618083164127

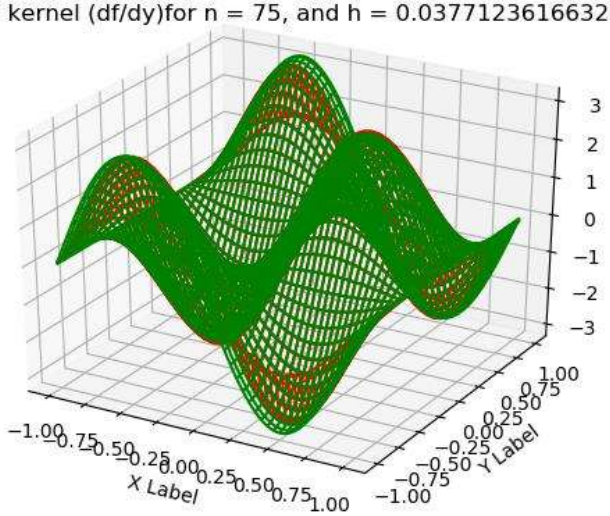


c) Its df/dy

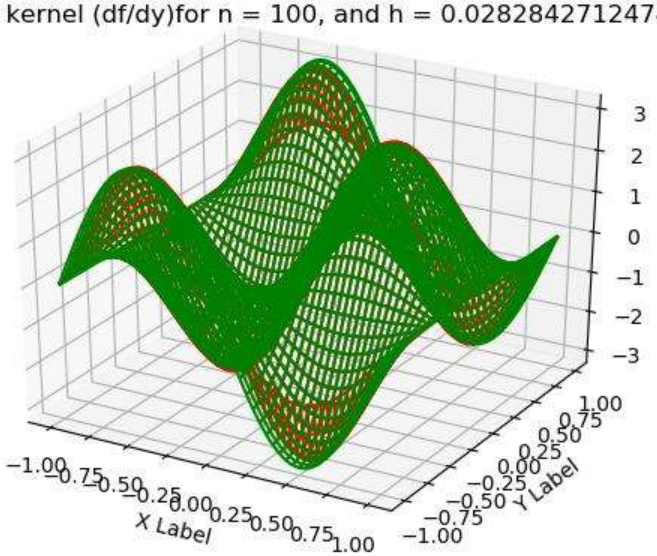

gaussian kernel (df/dy)for n = 25, and h = 0.1131370849898476
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
X Label
Y Label


gaussian kernel (df/dy)for n = 50, and h = 0.0565685424949238
3
2
1
0
−1
−2
−3
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
X Label
Y Label

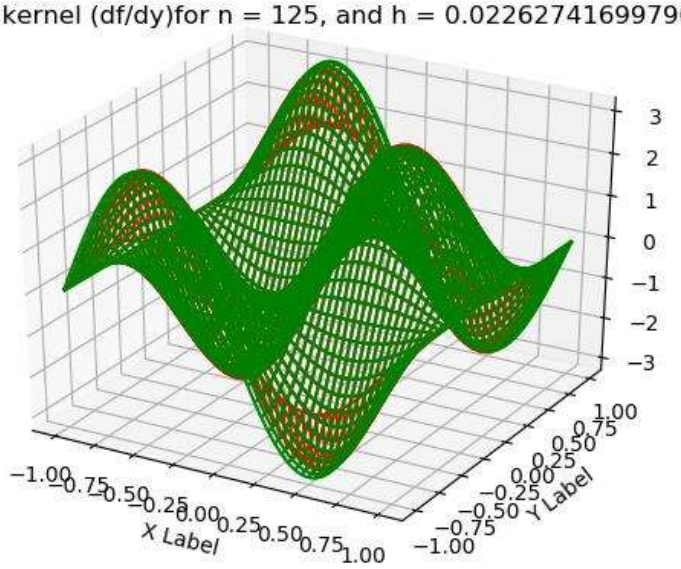
gaussian kernel (df/dy)for n = 75, and h = 0.037712361663282 54



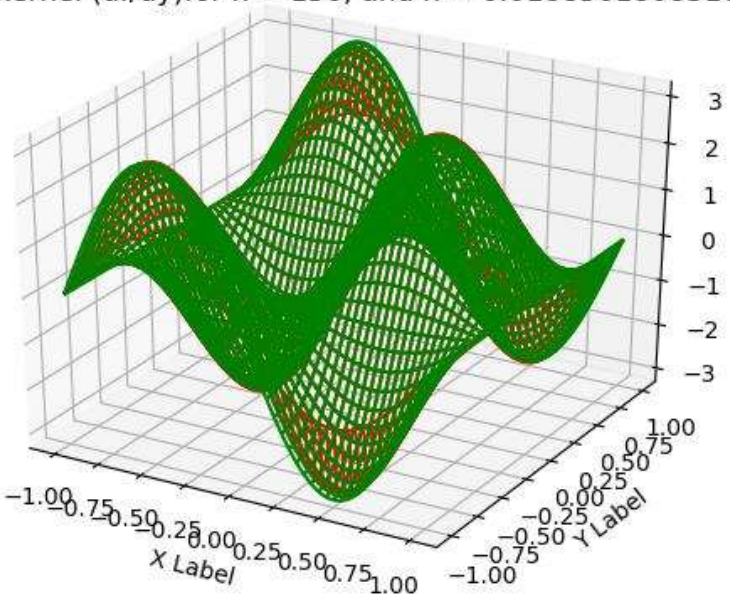
gaussian kernel (df/dy)for n = 100, and h = 0.0282842712474619



gaussian kernel (df/dy)for n = 125, and h = 0.02262741699796952

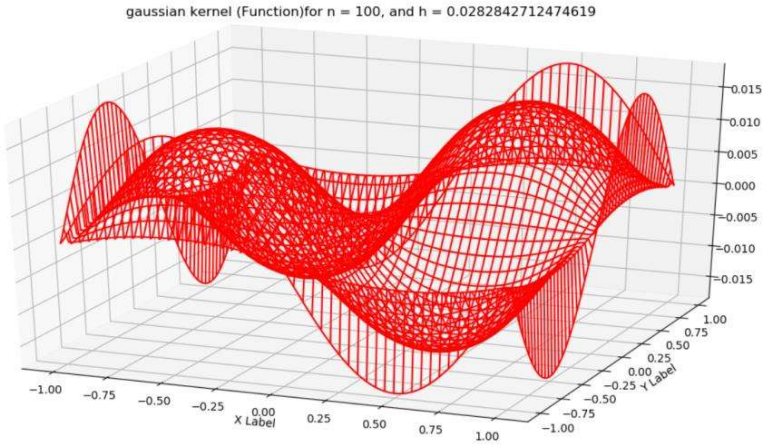


gaussian kernel (df/dy)for n = 150, and h = 0.01885618083164127

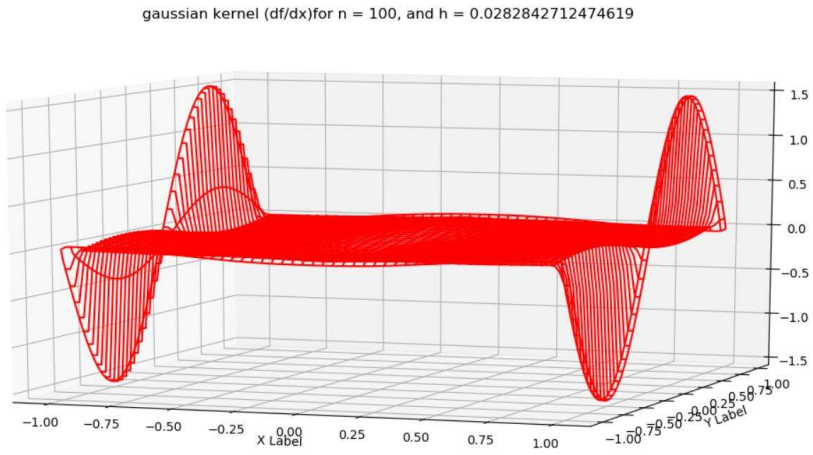


Now I have also plotted the difference between the Actual and Approximated function with and without boundary points

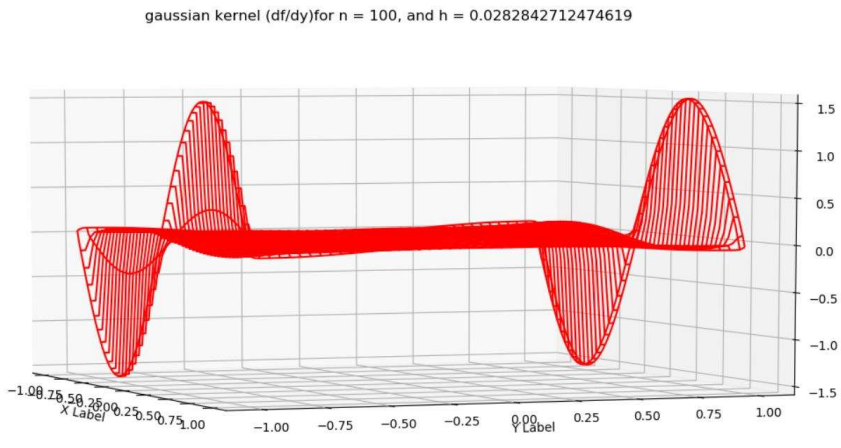
1) **With boundary points**
   a) **Function**



   b) **df/dx**



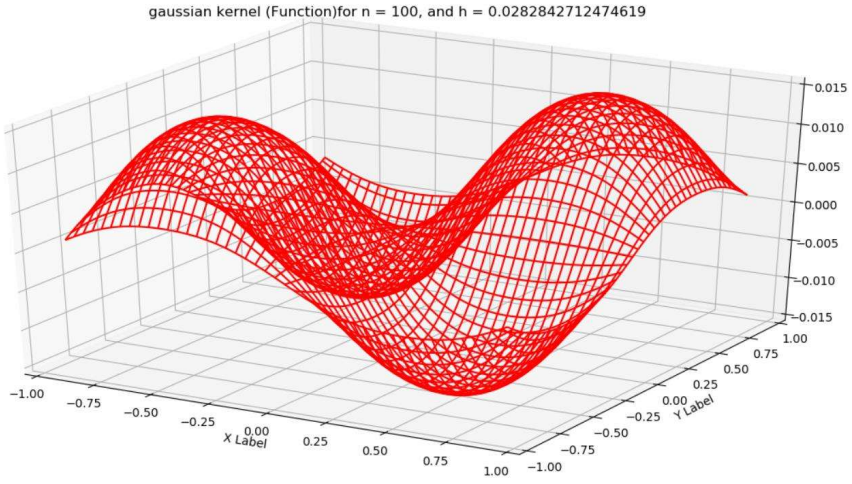
   c) **df/dy**



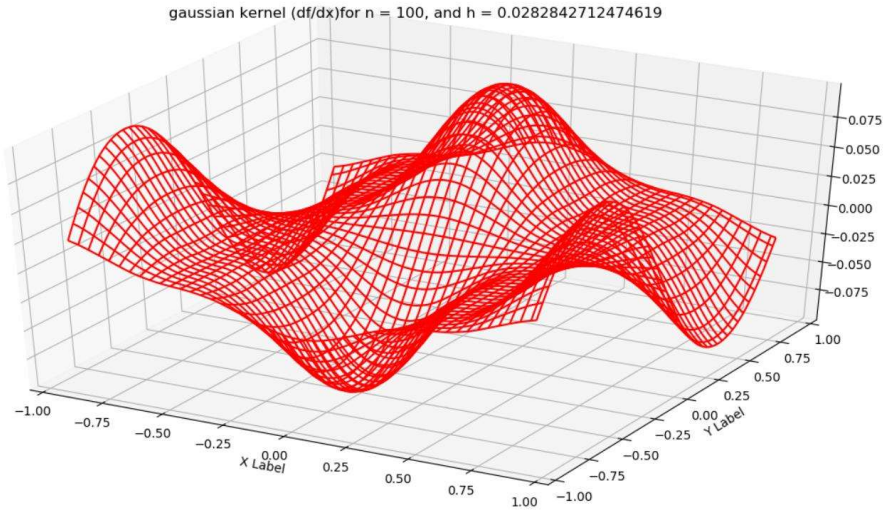
Kindly note the maximum error in derivatives at boundary only and we got a nice function approximation with order of difference around 0.015

2) Without boundary points

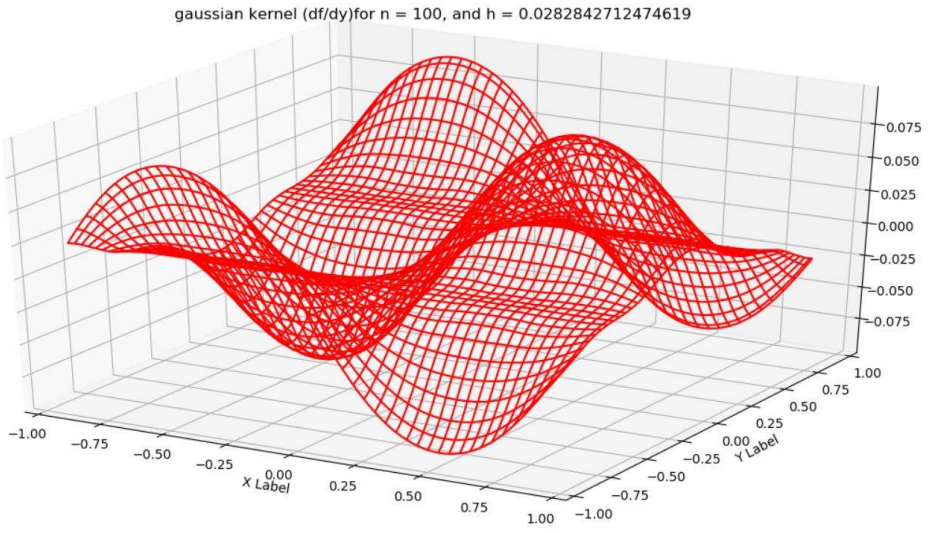
a) Function



b) df/dx

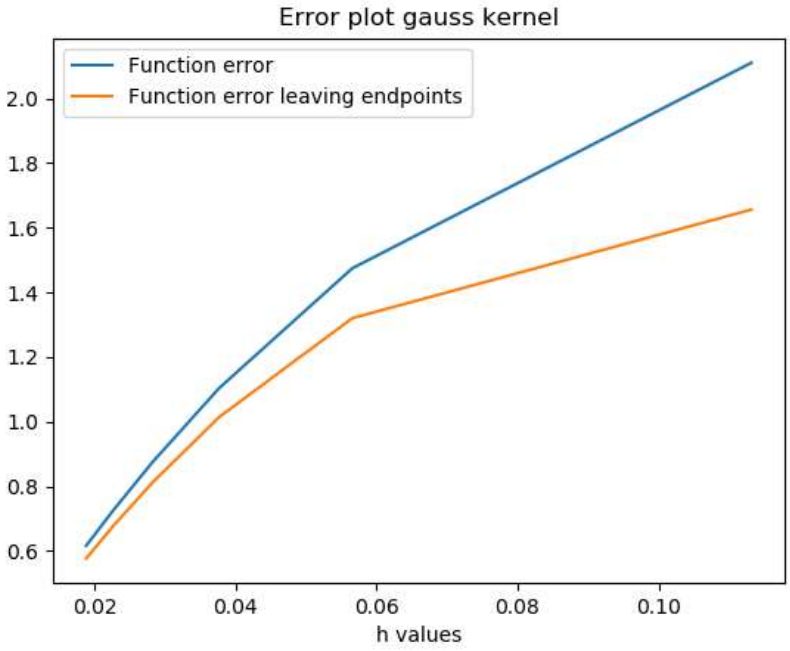


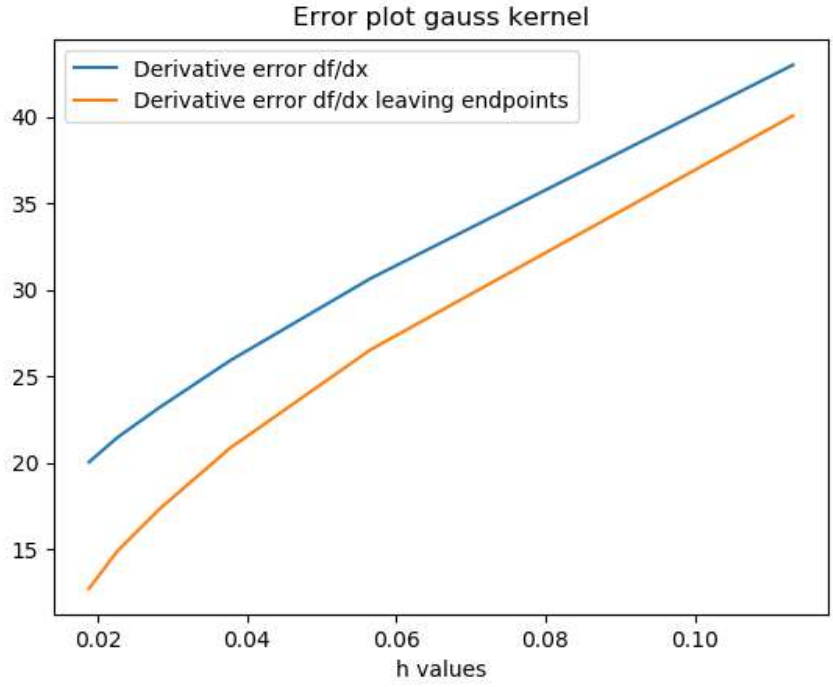
c) df/dy

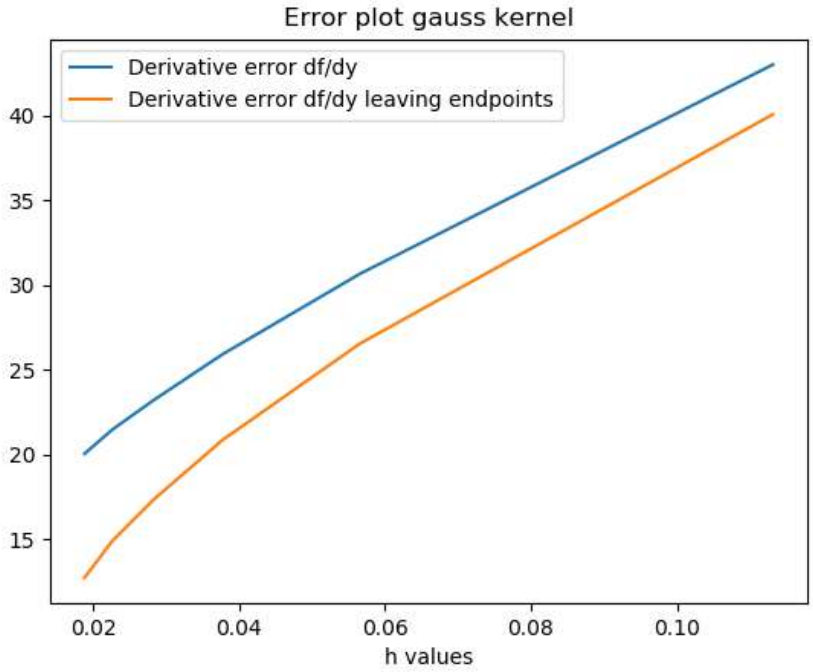


Note that after removing boundary points the error in the derivatives also comes to an order of 0.075

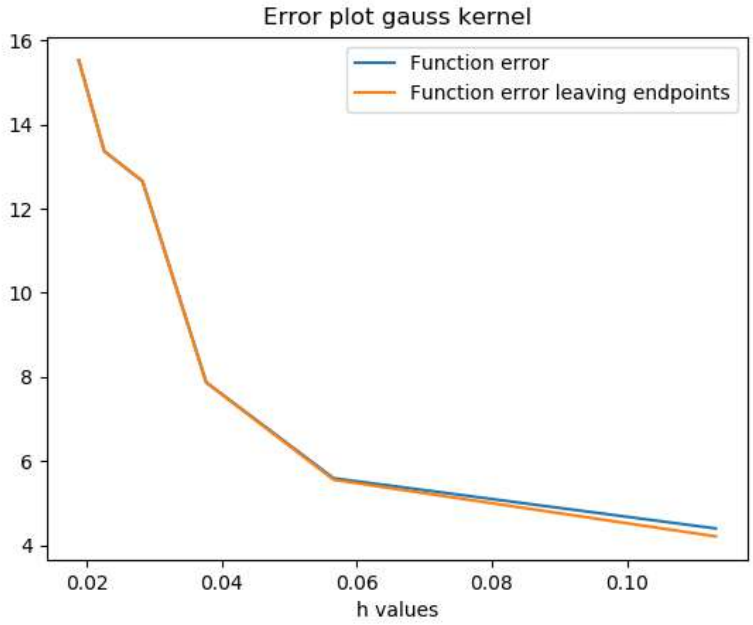
Error plots for Gaussian kernel without random noise

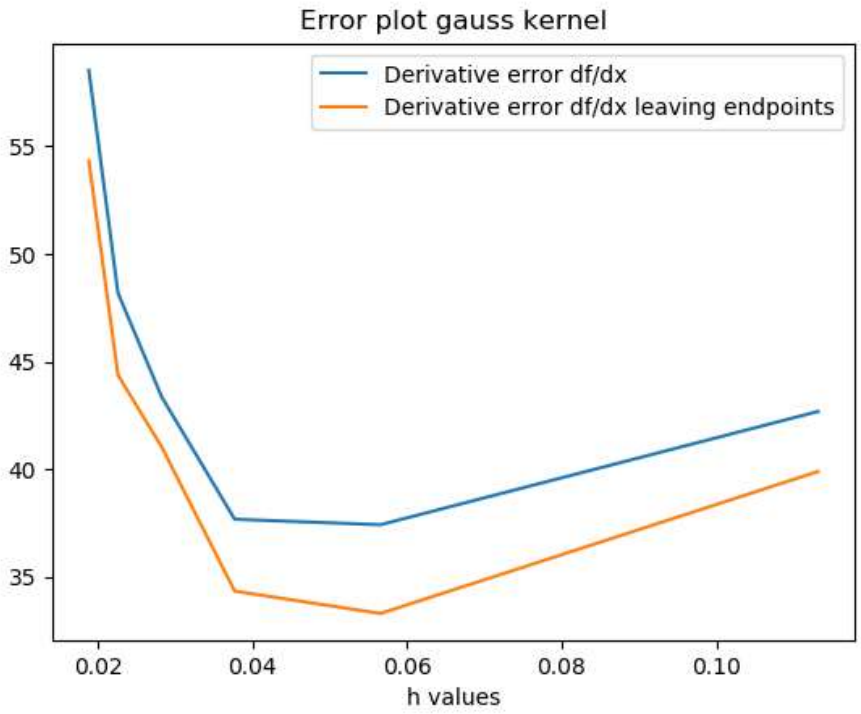


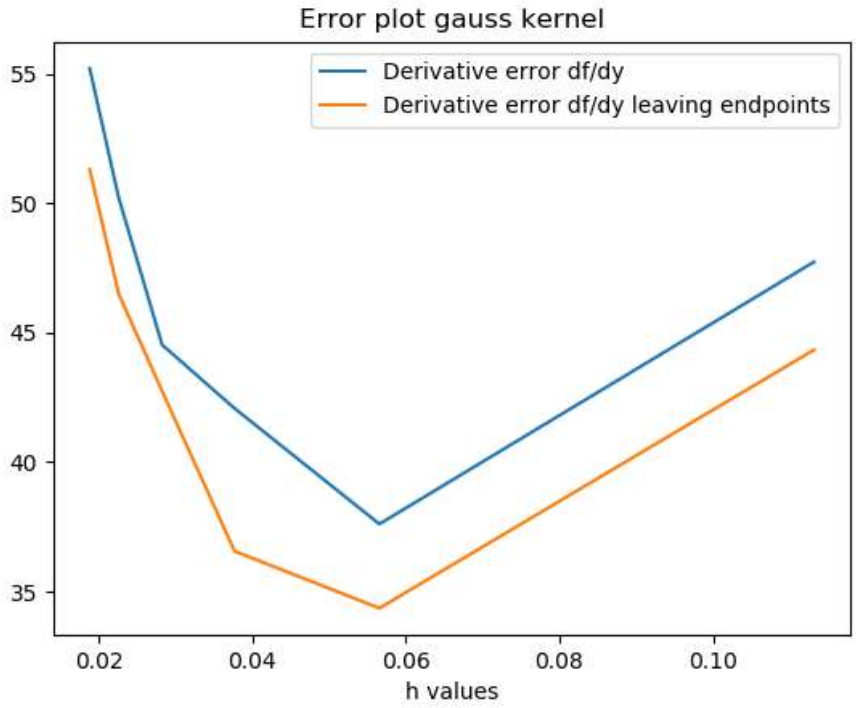




Error plots for Gaussian kernel with random noise

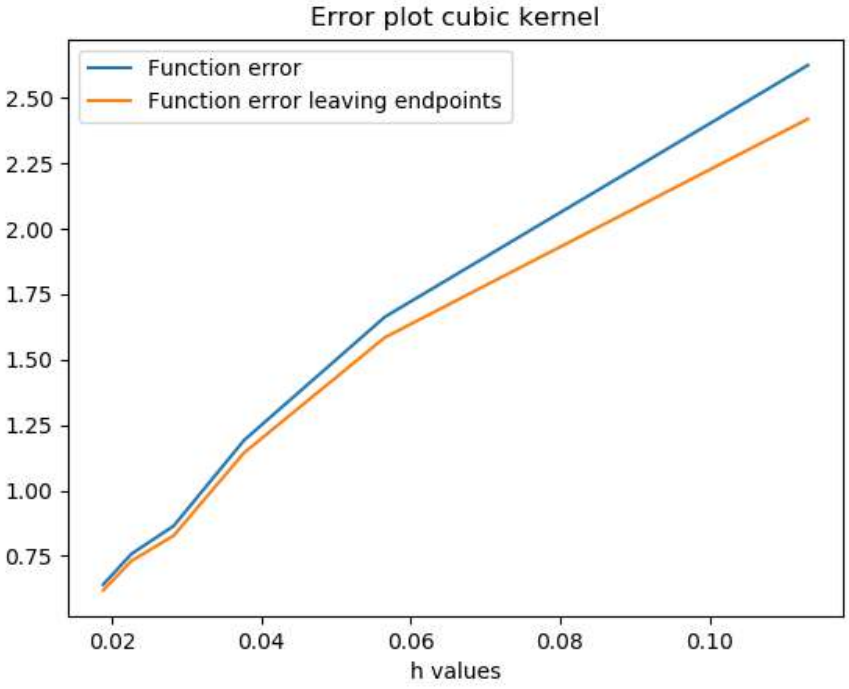


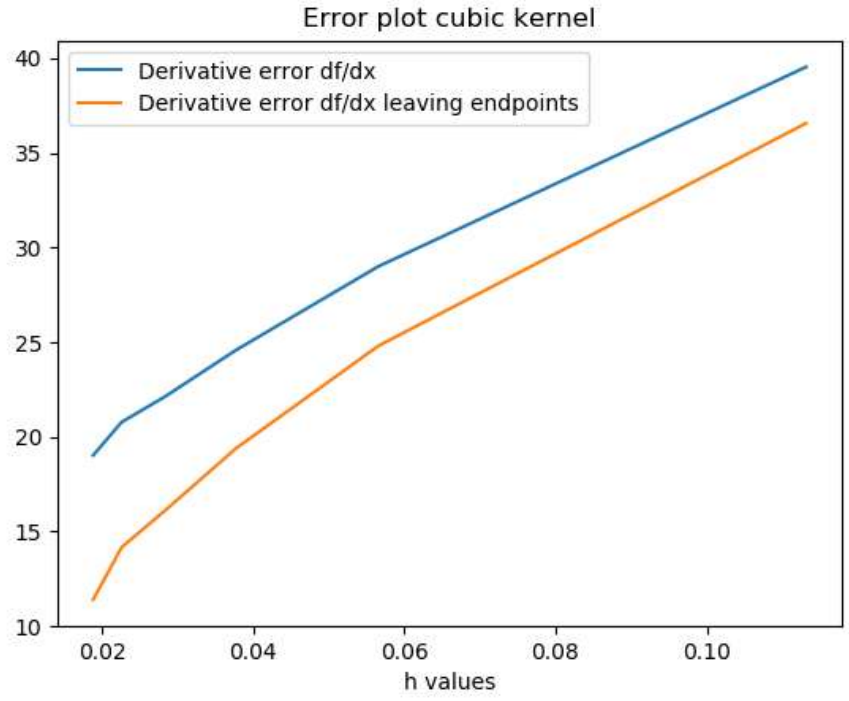

Error plot gauss kernel
Derivative error df/dx
Derivative error df/dx leaving endpoints
55
50
45
40
35
0.02
0.04
0.06
0.08
0.10
h values

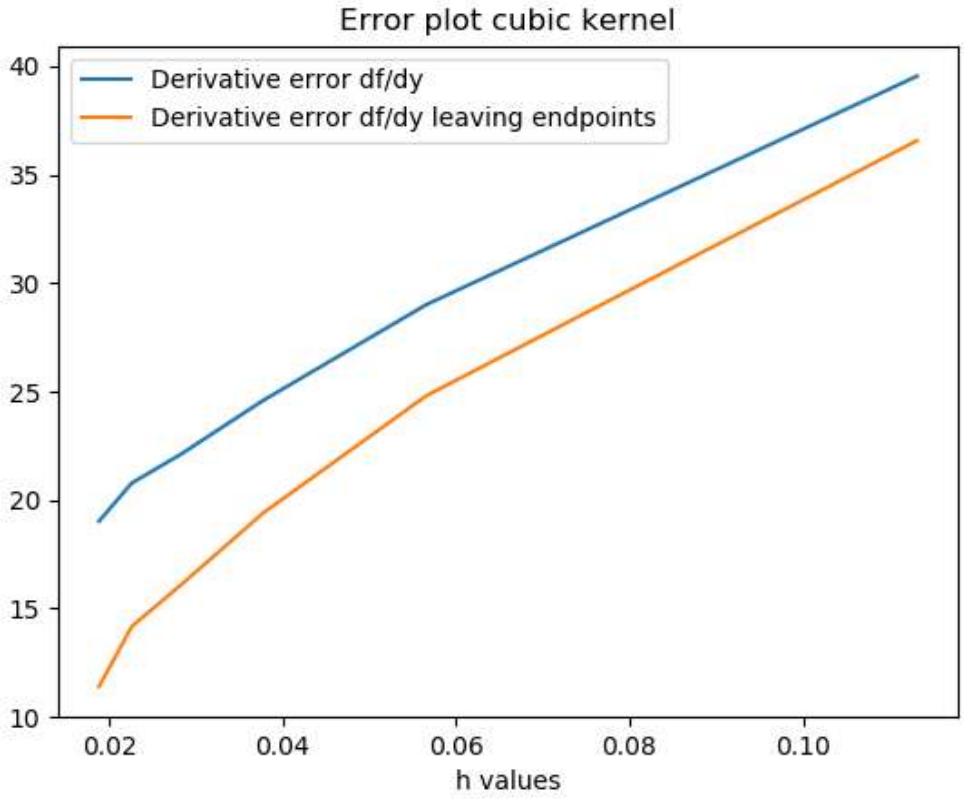

Error plot gauss kernel
Derivative error df/dy
Derivative error df/dy leaving endpoints
55
50
45
40
35
0.02
0.04
0.06
0.08
0.10
h values

2) Cubic kernels

Errors using Cubic kernel without random noise






Error plot cubic kernel
Derivative error df/dy
Derivative error df/dy leaving endpoints
40
35
30
25
20
15
10
0.02
0.04
0.06
0.08
0.10
h values

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.axes3d import Axes3D, get_test_data

def gauss_1d(ra,rb,h):
    q = np.abs((ra-rb)/h)
    alpha = 1.0/(np.sqrt(np.pi)*h)
    c_1 = np.where(q<=3)
    c_2 = np.where(q>3)
    ans = 0.0*np.asarray(q)
    ans[c_1] = alpha*np.exp(-q[c_1]*q[c_1])
    ans[c_2] = 0.0
    return ans

def grad_gauss_1d(ra,rb,h):
    q = (ra-rb)/h
    sgn_q = np.sign(q)
    q = np.abs(q)
    alpha = 1.0/(np.sqrt(np.pi)*h)
    c_1 = np.where(q<=3)
    c_2 = np.where(q>3)
    ans = 0.0*np.asarray(q)
    ans[c_1] = -2*q[c_1]*sgn_q[c_1]*alpha*np.exp(-q[c_1]*q[c_1])/h
    ans[c_2] = 0.0
    return ans

def gauss_2d(ra,rb,h):
    q = np.sqrt((ra[0]-rb[0])**2 + (ra[1]-rb[1])**2) /h
    alpha = 1.0/(np.pi*h**2)
    c_1 = np.where(q<=3)
    c_2 = np.where(q>3)
    ans = 0.0*np.asarray(q)
    ans[c_1] = alpha*np.exp(-q[c_1]*q[c_1])
    ans[c_2] = 0.0
    return ans

def grad_gauss_2d(ra,rb,h):
    q = np.sqrt((ra[0]-rb[0])**2 + (ra[1]-rb[1])**2) /h
    alpha = 1.0/(np.pi*h**2)
    c_1 = np.where(q<=3)
    c_2 = np.where(q>3)
    ans = np.zeros(shape=(np.shape(q)[0],np.shape(q)[1],2))
    ans[c_1[0],c_1[1],0] = -2*alpha*np.exp(-q[c_1]*q[c_1])*(ra[0]-rb[0])[c_1] / h**2
    ans[c_1[0],c_1[1],1] = -2*alpha*np.exp(-q[c_1]*q[c_1])*(ra[1]-rb[1])[c_1] / h**2
    ans[c_2] = np.array([0.0,0.0])
    return ans

def cubic_1d(ra,rb,h):
    q = np.abs((ra-rb)/h)
    alpha = 2.0/(3*h)
    c_1 = np.where(q<=1)
    c_2 = np.where((q<=2) & (q>1))
    c_3 = np.where(q>2)
    ans = 0.0*np.asarray(q)
    ans[c_1] = alpha*(1 - 1.5*q[c_1]**2 * (1 - q[c_1]/2))
    ans[c_2] = alpha*0.25*(2.0-q[c_2])**3
    ans[c_3] = 0.0
    return ans

def grad_cubic_1d(ra,rb,h):
    q = (ra-rb)/h
    sgn_q = np.sign(q)
    q = np.abs(q)
    alpha = 2.0/(3*h)
    c_1 = np.where(q<=1)
    c_2 = np.where((q<=2) & (q>1))
    c_3 = np.where(q>2)
```

```
    ans = 0.0*np.asarray(q)
    ans[c_1] = alpha*(-3*q[c_1] + (9/4)*q[c_1]**2)*sgn_q[c_1]  /h
    ans[c_2] = (3/4)*alpha*(2.0-q[c_2])**2 * (-sgn_q[c_2]) /h
    ans[c_3] = 0.0
    return ans

def cubic_2d(ra,rb,h):
    q = np.sqrt((ra[0]-rb[0])**2 + (ra[1]-rb[1])**2) /h
    alpha = 10.0/(7*np.pi*h**2)
    c_1 = np.where(q<=1)
    c_2 = np.where((q<=2) & (q>1))
    c_3 = np.where(q>2)
    ans = 0.0*np.asarray(q)
    ans[c_1] = alpha*(1 - 1.5*q[c_1]**2 * (1 - q[c_1]/2))
    ans[c_2] = alpha*0.25*(2.0-q[c_2])**3
    ans[c_3] = 0.0
    return ans

def grad_cubic_2d(ra,rb,h):
    q = np.sqrt((ra[0]-rb[0])**2 + (ra[1]-rb[1])**2) /h
    alpha = 10.0/(7*np.pi*h**2)
    c_1 = np.where(q<=1)
    c_2 = np.where((q<=2) & (q>1))
    c_3 = np.where(q>2)
    ans = np.zeros(shape=(np.shape(q)[0],np.shape(q)[1],2))

    ans[c_1[0],c_1[1],0] = alpha*(-3 + (9/4)*q[c_1])*(ra[0]-rb[0])[c_1] / h**2
    ans[c_1[0],c_1[1],1] = alpha*(-3 + (9/4)*q[c_1])*(ra[1]-rb[1])[c_1] / h**2

    ans[c_2[0],c_2[1],0] = (-3/4)*alpha*(2.0-q[c_2])**2 *(ra[0]-rb[0])[c_2] / (q[c_2]*h**
    2)
    ans[c_2[0],c_2[1],1] = (-3/4)*alpha*(2.0-q[c_2])**2 *(ra[1]-rb[1])[c_2] / (q[c_2]*h**
    2)

    ans[c_3] = np.array([0.0,0.0])

    return ans

def plot_kernels_1d(n):
    x = np.linspace(-1, 1.0, 100)
    fx = np.sin(np.pi*x)
    der_fx = np.pi*np.cos(np.pi*x)
    dx = 2.0/n
    xj = np.linspace(-1.0+0*dx, 1.0-0*dx, n)
    #xj = xj + np.random.normal(0,0.01,n)
    pho = np.ones(np.shape(xj)[0])
    mass = pho*dx
    fj = np.sin(np.pi*xj)
    h = 1.0*dx
    res = 0.0
    #plt.figure(1)
    #plt.plot(x, np.sin(np.pi*x),'g',label='Function')
    res1 = 0.0
    for j in range(n):
        res += dx*fj[j]*cubic_1d(x, xj[j], h)
        res1 += dx*( fj[j] - np.sin(np.pi*x) )*grad_cubic_1d(x, xj[j], h)
    #plt.plot(x, res, 'r*',label='function Approximation')
    #plt.plot(x,res1,'k',label='Derivative Approximation')
    #plt.plot(x,np.pi*np.cos(np.pi*x),'y*',label='Derivative')
    #plt.legend(loc='best')
    #plt.title('gaussian kernel for n = '+str(n)+', and h = '+str(h))
    #plt.savefig('gauss_'+str(n)+'.png')
    #plt.close()
    e_1 = np.sqrt(np.sum(np.square(res-fx)))
    e_2 = np.sqrt(np.sum(np.square(res-fx)[1:-1]))
    e_3 = np.sqrt(np.sum(np.square(res1-der_fx)))
    e_4 = np.sqrt(np.sum(np.square(res1-der_fx)[1:-1]))
```

```
133         return h,e_1,e_2,e_3,e_4
134
135     def plot_kernels_2d(n):
136         x = np.linspace(-1, 1.0, 100)
137         y = np.linspace(-1, 1.0, 100)
138         X,Y = np.meshgrid(x,y)
139         Z = np.sin(np.pi*X)*np.sin(np.pi*Y)
140         der_Zx = np.pi*np.cos(np.pi*X)*np.sin(np.pi*Y)
141         der_Zy = np.pi*np.sin(np.pi*X)*np.cos(np.pi*Y)
142
143         dx = 2.0/n
144         dy = 2.0/n
145         dr = (dx**2 + dy**2)**0.5
146         xj = np.linspace(-1.0+1*dx, 1.0-1*dx, n)
147         yj = np.linspace(-1.0+1*dy, 1.0-1*dy, n)
148
149         #xj += np.random.normal(0, 0.01, n)
150         #yj += np.random.normal(0, 0.01, n)
151
152         pho = np.ones(shape=(np.shape(xj)[0],np.shape(yj)[0]))
153         mass = pho*dr
154         Xj , Yj = np.meshgrid(xj,yj)
155         fj = np.sin(np.pi*Xj)*np.sin(np.pi*Yj)
156         h = 1.0*dr
157
158         res = 0.0 #for the function
159         res1 = 0.0 # for the x derivative
160         res2 = 0.0 # for the y derivative
161
162         for i in range(n):
163             for j in range(n):
164                 res += (0.5*dr**2)*fj[i,j]*cubic_2d([X,Y],[Xj[i,j],Yj[i,j]], h)
165                 res1 += (0.5*dr**2)*( fj[i,j] - Z )*grad_cubic_2d([X,Y],[Xj[i,j],Yj[i,j]], h
                    )[:,:,0]
166                 res2 += (0.5*dr**2)*( fj[i,j] - Z )*grad_cubic_2d([X,Y],[Xj[i,j],Yj[i,j]], h
                    )[:,:,1]
167
168         e_1 = np.sqrt(np.sum(np.square(res-Z)))
169         e_2 = np.sqrt(np.sum(np.square(res-Z)[1:-1,1:-1]))
170         e_3 = np.sqrt(np.sum(np.square(res1-der_Zx)))
171         e_4 = np.sqrt(np.sum(np.square(res1-der_Zx)[1:-1,1:-1]))
172         e_5 = np.sqrt(np.sum(np.square(res2-der_Zy)))
173         e_6 = np.sqrt(np.sum(np.square(res2-der_Zy)[1:-1,1:-1]))
174
175         return h,e_1,e_2,e_3,e_4,e_5,e_6
176
177         """
178         plt.figure(1)
179         ax = plt.axes(projection = '3d')
180         ax.plot_wireframe(X, Y, Z,label='original',color = 'g')
181         ax.plot_wireframe(X, Y, res,color = 'r')
182         plt.title('gaussian kernel (Function)for n = '+str(n)+', and h = '+str(h))
183         ax.set_xlabel('X Label')
184         ax.set_ylabel('Y Label')
185         plt.savefig('2d_gauss_1'+str(n)+'.png')
186         plt.close()
187
188         plt.figure(1)
189         ax = plt.axes(projection = '3d')
190         ax.plot_wireframe(X, Y, der_Zx,color = 'g')
191         ax.plot_wireframe(X, Y, res1,color = 'r')
192         plt.title('gaussian kernel (df/dx)for n = '+str(n)+', and h = '+str(h))
193         ax.set_xlabel('X Label')
194         ax.set_ylabel('Y Label')
195         plt.savefig('2d_gauss_2'+str(n)+'.png')
196         plt.close()
197
```

```
    plt.figure(1)
    ax = plt.axes(projection = '3d')
    ax.plot_wireframe(X, Y, der_Zy,color = 'g')
    ax.plot_wireframe(X, Y, res2,color = 'r')
    plt.title('gaussian kernel (df/dy)for n = '+str(n)+', and h = '+str(h))
    ax.set_xlabel('X Label')
    ax.set_ylabel('Y Label')
    plt.savefig('2d_gauss_3'+str(n)+'.png')
    plt.close()
    """

def save_plots():
    for iter_1 in range(1,7):
        plot_kernels_2d(iter_1*25)

def error_plots():
    ha=[]
    e1a=[]
    e2a=[]
    e3a=[]
    e4a=[]
    e5a=[]
    e6a=[]
    for iter_1 in range(1,7):
        h,e_1,e_2,e_3,e_4,e_5,e_6 = plot_kernels_2d(iter_1*25)
        ha.append(h)
        e1a.append(e_1)
        e2a.append(e_2)
        e3a.append(e_3)
        e4a.append(e_4)
        e5a.append(e_5)
        e6a.append(e_6)

    plt.figure(1)
    plt.plot(ha,e1a,label='Function error')
    plt.plot(ha,e2a,label='Function error leaving endpoints')
    plt.legend(loc='best')
    plt.title('Error plot cubic kernel')
    plt.xlabel('h values')
    plt.savefig('2d_cubic_error_1')
    plt.close()

    plt.figure(1)
    plt.plot(ha,e3a,label='Derivative error df/dx')
    plt.plot(ha,e4a,label='Derivative error df/dx leaving endpoints')
    plt.legend(loc='best')
    plt.title('Error plot cubic kernel')
    plt.xlabel('h values')
    plt.savefig('2d_cubic_error_2')
    plt.close()

    plt.figure(1)
    plt.plot(ha,e5a,label='Derivative error df/dy')
    plt.plot(ha,e6a,label='Derivative error df/dy leaving endpoints')
    plt.legend(loc='best')
    plt.title('Error plot cubic kernel')
    plt.xlabel('h values')
    plt.savefig('2d_cubic_error_3')
    plt.close()

error_plots()
#save_plots()
#plot_kernels_1d(100)
#plot_kernels_2d(75)
#plt.show()
```